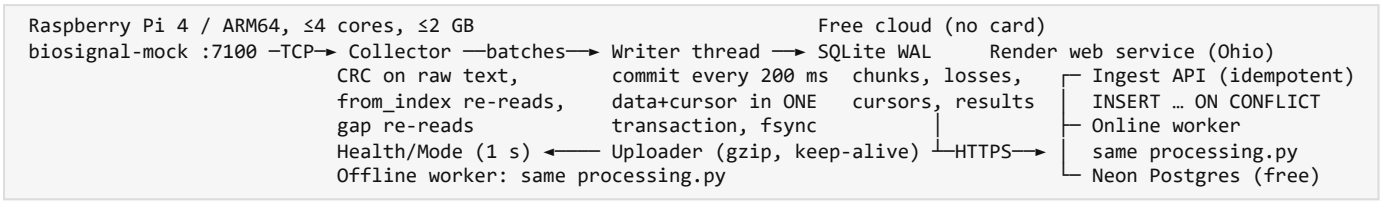
Never Lose a Heartbeat: design note

hbpipe · code: github.com/Harsh202458/mayacardio-hbpipe · cloud: hbpipe-cloud.onrender.com · device evidence: ARM64 (linux/arm64 Debian 12 container, --cpus 4 --memory 2g)

1. Architecture



- **Collector**: one TCP client polls each sensor every 100 ms and always requests `from_index = cursor`, so restarts, duplicate and corrupt replies are harmless re-reads. The CRC-32 is checked on the *exact* `data` text received (never re-serialised JSON). Every line is applied by its own sensor and index range.
- **Writer thread**: commits batches strictly in order, so a slow fsync (SD cards stall) never stops reading the mock's 10 s buffer.
- **Uploader**: offline results first, then the newest 30 s (live lane), then the backlog oldest-first. Data counts as uploaded only after the cloud has committed and replied. Progress is kept as acknowledged chunk-id ranges, so stored rows are never rewritten.
- **Mode**: health ping every 1 s on a kept-alive connection. Offline after ≥2 failures and 5 s without success; online after 2 successes (pings every 0.5 s while offline). systemd `Restart=always, RestartSec=200ms`; the collector starts before the cloud code even loads.
- **Processing**: one module, used unchanged online and offline. Window *w* is computed only when all its inputs are final (data, recorded loss, or past session end), so samples in transit are never counted as missing. The cloud skips windows the device already processed offline.

2. Why no sample is lost silently

For every (session, sensor), the indices `[0, cursor)` are covered **exactly once** by stored blocks and recorded losses. Each block or loss is committed in the same transaction as its cursor advance, so this holds after any crash.

- **Uncommitted data is simply "not read yet":** it is re-read after a restart while the mock still holds it.
- **Unexplained index jump:** re-read; recorded as `not_delivered_by_mock` only after 5 failed re-reads.
- **New `session_id`:** the old session's unread tail is listed with `to_index: null`, reason `device_power_loss_or_reboot` (boot id changed) or `mock_restart`.
- **Disk full:** release a 16 MB reserve and keep saving; below 4 MB, store only `disk_full` loss records while still reading the mock; resume when space returns.
- **Exactly once in the cloud:** immutable blocks keyed (session, sensor, first_index) plus ack-after-commit, so re-sends are no-ops.
- **Nothing is ever interpolated.**

3. Guaranteed / not guaranteed

Guaranteed	Not guaranteed
Every sample received from the mock is stored once, unchanged (CRC + range checks)	Samples the mock never delivered (listed with the mock's reason)
Every index not stored is listed (sensor, range, reason); export = samples + losses	The last ≤ 0.5 s before a power cut (listed, not silent)
Process crash / `kill -9`: resume from the last commit inside the mock's 10 s buffer	Restarts slower than 10 s (seen only under ARM emulation; listed as `buffer_overflow`)
Mode switch ≤ 10 s each way; no sample lost or duplicated by a switch	Storage that acknowledges a flush it did not perform
Same results online and offline (one code path, exact rational rounding)	Outages longer than free disk (~263 MB/h written)
System clock changes have no effect (index ordering, monotonic timers)	Render free tier cold start (~1 min) if idle > 15 min

4. Maximum data loss on a power cut: 0.5 s per sensor

When power is cut, the mock dies too. What can be lost is what the mock had produced but we had not yet committed:

- up to one poll interval (0.1 s);
- plus one commit interval (0.2 s);
- plus the fsync time.

That is ≈ 0.35 s; we state **0.5 s**. Measured on the ARM64 device (whole container killed, mock included): **0.02–0.055 s per sensor over 2 cuts**. These samples appear after reboot as the old session's tail loss (`to_index: null`). Committing more often lowers the bound at the cost of SD-card writes.

5. Results logic (section 3)

- **Windows:** `400w…400w+399`. Missing counts, per-channel mean/min/max over available samples (`null` if none); magnitude = mean $\sqrt{(ch1^2+ch2^2)}$ of sensor 2.
- **Peaks:** the 800 previous samples and the next one are available, strict local maximum, `2·x > max(previous 800)`, ≥100 after the previous peak (rolling max via a monotonic deque, O(n)).
- **Heart rate:** `24000·(k−1)/(last−first)` over the peaks in the 5 s span; `null` if w<4, any sample is missing, or <2 peaks. The chain is warmed up 2000 samples earlier, which matches the full-history chain (unit-tested).
- **Rounding:** half away from zero, on exact rationals.
- **Check:** the ECG mock gives 60/90/72 bpm exactly as scheduled.

6. Measured (ARM64 device, final code)

Check	Result
6× `kill -9` + mock faults (corrupt, duplicate, stalls, dropout)	PASS: device = cloud, no silent loss, 330/330 windows
Network cut 30 s / 3 min	PASS: offline after 4.8 / 3.8 s, online after 0.3 / 0.9 s
2 power cuts (device + mock killed)	PASS: lost 0.02–0.055 s per sensor, listed as `to_index: null`
Disk full (64 MB fs filled in 2 stages, then freed)	PASS: kept saving into the reserve; `disk_full` losses only when full; recovered
Clock +1 h, −2 h, +1 day	PASS: 0 samples lost
2 h outage catch-up (5.76 M samples)	PASS: cloud complete in 2.9 min (limit 15)
Results (ECG)	HR 60 / 90 / 72 bpm as scheduled; online = offline

7. Resources, 1 h online against the deployed cloud

Avg CPU (all 4 cores)	Peak memory	Disk written	Network sent
5.7 % pipeline · 15.8 % whole device (incl. mock)	83 MB pipeline · 252 MB whole device	263 MB / h (whole device)	27.3 MB / h

Environment: linux/arm64 Debian 12 container (QEMU-emulated on an x86-64 Windows PC), --cpus 4 --memory 2g; cloud Render free (Ohio) + Neon free; device in India. 2.92 M samples streamed; online latency over the internet: median 1.1 s, p95 4.3 s, 96 % of windows within 5 s. The slower windows and 13 brief `buffer_overflow` gaps (2,478 samples, all listed) occurred only in short bursts while a second emulated device ran on the same PC to record the demo video. Emulation makes CPU figures pessimistic. Natively, the pipeline goes from process start to reading the mock in 0.13–0.36 s.

8. Assumptions and notes

- **Official mock:** the mock download link was blank in our copy. We implemented section 6 faithfully (`tools/fake_mock.py`) plus a ground-truth checker; the pipeline uses only the documented protocol.
- **Assumed:** the peak chain carries across the session; stats fields are `mean_1/min_1/max_1/…` per-channel lists; windows processed offline are not recomputed online.
- **Free services only:** Render free + Neon free, with no payment method on either. Everything else is open source.